COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	October 15, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Asajar, Carlo Mari Godino, Jaime Jr.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 13.1 PARALLEL ALGORITHMS AND MULTITHREADING

1. PROCEDURE OUTPUT

ILO A & ILO B:

Part 1: Simple One-Threaded Example

Since all threads must be given a function to complete at their creation, we first must declare a function for it to be given. We'll name this function print and will design it to take int and string arguments when called. When executed, this code will simply report the data values passed in.

In the next section, we'll initialize a thread and have it execute the above function. To do this, we'll have the main function, the default executor present in all C++ applications, initialize the thread for the print function. After that, we use another handy multithreading command, join(), pausing the main function's thread until the specified thread, in this case t1, has finished its task. Without join() here, the main thread would finish its task before t1 would complete print, resulting in an error.

Show your output and analysis in the table.

Run this code and show the output in the table.
(Table 13-1. Simple One-Threaded Example)

Screenshot of Code	**Code: (main_Part1.cpp)** PROCEDURE > main_Part1.cpp > ... 1 #include <iostream> 2 #include <thread> 3 4 void print(int n, const std::string &str) {

<table>
<tr><td></td><td>

```
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();

    return 0;
}
```

</td></tr>
<tr><td>Asajar's Output</td><td>

```
Printing integer: 10
Printing string: T.I.P.
```

</td></tr>
<tr><td>Godino's Output</td><td>

```
Printing integer: 10
Printing string: T.I.P.
```

</td></tr>
<tr><td>Analysis</td><td>The code successfully demonstrated how multithreading works in C++ by creating a separate thread to perform a specific task. The print() function was used to display the integer value 10 and the string "T.I.P.", while the std::thread created a new thread and passed the needed values to the function. The t1.join() statement made the main thread wait until the created thread finished its task before the program continued. The output showed the correct integer and string values, which means that the function was executed properly by the created thread. Overall, the print() function, std::thread, and join() method worked together to complete the task correctly and demonstrate the basic use of multithreading.</td></tr>
</table>

Table 13-1. Simple One-Threaded Example

Part 2: Multi-Threaded Example

While the outcome of the single thread example above could easily be replicated without using multithreaded code, we can truly see concurrency's benefits when we attempt to complete print multiple times with different sets of data. Without multithreading, this would be done by simply having the main thread repeat print one at a time until completion.

To do this with concurrency in mind, we instead use a for loop to initialize multiple threads, pass them the print function and arguments, which they then complete concurrently. This multithreading option would be faster using only the main thread as more of the total CPU is being used.

Runtime difference between multithreading and non-multithreading solutions increasing as more print executions are needed.

Let's see what a many-thread version of the above code would look like

Show your output and analysis in the table.
(Table 13-2. Multithreaded Example)

Screenshot of Code	**Code: (main_Part2.cpp)**
	PROCEDURE > main_Part2.cpp > ...
	1 #include <iostream>
	2 #include <thread>
	3 #include <vector>
	4 #include <string>
	5
	6 void print(int n, const std::string &str) {
	7 std::string msg = std::to_string(n) + " : " + str;
	8 std::cout << msg << std::endl;
	9 }
	10
	11 int main() {
	12
	13 std::vector<std::string> s = {
	14 "T.I.P.",
	15 "Competent",
	16 "Computer",
	17 "Engineers"

```
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

Asajar's Output

First Output:

```
0 : T.I.P.2 : Computer

3 : Engineers1 : Competent
```

Second Output:

```
0 : T.I.P.2 : Computer

1 : Competent
3 : Engineers
```

Godino's Output	**First Output:**
	`0 : T.I.P.3 : Engineers1 : Competent`
	`2 : Computer`
	Second Output:
	`0 : T.I.P.`
	`1 : Competent`
	`3 : Engineers`
	`2 : Computer`

Analysis	The code successfully demonstrated how ***multithreading*** works in C++ by creating multiple threads to print different sets of data ***concurrently***. The ***std::vector threads;*** statement was used to store the created threads, while the ***for loop*** and ***threads.push_back(std::thread (print, i, s[i]));*** allowed the program to create four threads with different integer and string values. The ***print() function*** combined each number and its matching string into one message before displaying it, which helped keep the output together. The important part was the ***th.join()*** statement because it made the main thread wait until each created thread finished its task before the program ended. During the execution, the ***output order*** could change every time because the threads were running concurrently, but all the numbers and their matching strings were still displayed correctly. Through this activity, it became easier to understand how ***multiple threads, concurrent execution,*** and the ***join() method*** work together, although multithreading may not always be faster for simple tasks like printing a few lines.

Table 13-2. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. **Write a definition of multithreading and its advantages/disadvantages.**

Asajar's Answer	Multithreading is a programming technique that allows a program to use multiple threads to handle different tasks during overlapping periods of time. It can help make a program more responsive and use the CPU better, especially when different processor cores are available. One advantage is that several tasks can make progress without waiting for one another all the time. However, it can also make the code harder to manage because threads may access the same data and cause problems if they are not handled properly. In the activity, std::thread t1(print, 10, "T.I.P."); created a separate thread, while t1.join(); made the main thread wait until its task was finished. This showed that multithreading can help organize tasks, but proper thread management is still needed to avoid errors (GeeksforGeeks, 2026; cppreference.com, n.d.).
Godino's Answer	Multithreading allows a single program to run multiple threads that can work independently while sharing the same memory space. Its main benefits include improved performance, better responsiveness, and more efficient use of available processor resources. It is useful when applications need to handle several activities without making the whole program wait. However, it also has disadvantages, such as race conditions, deadlocks, and additional work in synchronizing shared data. These problems can make programs more difficult to test and debug. Therefore, multithreading can improve an application's efficiency, but it requires careful planning and control to ensure that each thread works correctly (GeeksforGeeks, 2026).

2. **Rationalize the use of multithreading by providing at least 3 use-cases.**

Asajar's Answer	Multithreading is useful when a program needs to handle different activities without making the user wait for every task to finish first. First, in web browsers, it can help users download files, play videos, and browse websites while still interacting with the application. Second, in video editing and image processing, multiple threads can work on different parts of a file, which may reduce the time needed to finish the process. Third, in online games, different threads can help manage player input, sound, and game logic so the game can respond more smoothly. In the C++ activity, std::vector<std::thread> threads; stored multiple threads, while the for loop created threads to print different values. This showed how several tasks can make progress concurrently, although the actual speed improvement still depends on the type of work and the computer's available resources (GeeksforGeeks, 2026).

Godino's Answer	Multithreading is commonly applied in software that must perform several operations efficiently. One example is web servers, which can use multiple threads to handle requests from different users. Another example is scientific computing, where calculations can be divided into smaller tasks that may run in parallel on different processor cores. A third example is media applications, which can process audio or video while continuing to respond to user commands. These examples show how multithreading can support responsiveness and performance when tasks can be handled independently. However, the benefits depend on the workload, hardware, and extra time needed to manage threads (GeeksforGeeks, 2026).

3. Differentiate between parallelism and concurrency.

Asajar's Answer	Concurrency means that multiple tasks can make progress during overlapping periods of time, even when they take turns using only one CPU core. Meanwhile, parallelism happens when multiple tasks are actually processed at the same time, usually with the help of multiple CPU cores. The main difference is that concurrency focuses on managing different tasks, while parallelism focuses on executing tasks simultaneously. For example, switching between downloading a file and playing music shows concurrency, while using multiple processor cores to calculate different parts of a large dataset shows parallelism. In the C++ activity, std::thread created multiple threads, but that did not guarantee that all of them would run at exactly the same time. This helped show that concurrency and parallelism are related but not the same, because their actual execution depends on how the computer handles the tasks (GeeksforGeeks, 2026).
Godino's Answer	Concurrency and parallelism are two ways of handling multiple tasks in a computer program, but they differ in how the tasks are executed. Concurrency allows tasks to make progress within overlapping time periods through task switching or simultaneous execution. Parallelism specifically refers to multiple tasks being executed simultaneously, commonly through multiple processor cores. For instance, an application that switches between background downloads and user input demonstrates concurrency, while a program dividing a large calculation among several CPU cores demonstrates parallelism. Understanding these concepts helps developers select a suitable approach based on the application's needs and available hardware (GeeksforGeeks, 2026).

References	cppreference.com. (n.d.). *std::thread*. https://en.cppreference.com/w/cpp/thread/thread GeeksforGeeks. (2026, May 15). *Multithreading in C++*. https://www.geeksforgeeks.org/cpp/multithreading-in-cpp/ GeeksforGeeks. (2026, May 4). *Difference between concurrency and parallelism*. https://www.geeksforgeeks.org/operating-systems/difference-between-concurrency-and-parallelism/

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

- Create a global variable of type integer.
- Create an add function that will take an integer parameter and add that value to the global variable.
- Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
- Display the value of global variables at different combinations of using the join() per thread. Such that:
    - Display
    - T1.join()
    - Display
    - T2.join()
    - Display
    - T3.join()

Provide an analysis based on the outputs of the multi-threading exercise

Screenshot of the Code	**My Code: (main_PartB.cpp)** SUPPLEMENTARY > main_PartB.cpp > main() 1 #include <iostream> 2 #include <thread> 3 #include <mutex> 4 #include <chrono> 5 6 int globalValue = 0; 7 std::mutex mtx; 8 9 // Add a value to the global variable 10 void add(int value){ 11

```
    // Prevent simultaneous access to the global variable
    std::lock_guard<std::mutex> lock(mtx);
    globalValue += value;
}

// Display the global variable
void display(){

    // Prevent simultaneous access while displaying
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << "Global Variable: " << globalValue << std::endl;
}

// Test the execution time of multithreading
void testMultithreading(int testNumber){

    // Reset the global variable
    globalValue = 0;

    // Start the timer
    auto begin = std::chrono::steady_clock::now();

    // Create three threads
    std::thread T1(add, 100);
    std::thread T2(add, 200);
    std::thread T3(add, 300);

    // Wait for all threads to finish
    T1.join();
    T2.join();
    T3.join();

    // Stop the timer
    auto end = std::chrono::steady_clock::now();

```

```
    // Display the test number
    std::cout << "\nTest Number: " << testNumber << std::endl;

    // Display the execution time
    std::cout << "Execution Time: "
              << std::chrono::duration_cast<
                 std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    // Display the final value
    std::cout << "Final Value: " << globalValue << std::endl;
}
```

```
59
60  int main(){
61
62      // Display the algorithm name
63      std::cout << "\nMultithreading:";
64      std::cout << "\n-----------------------------------------------";
65
66      // Create three threads
67      std::thread T1(add, 100);
68      std::thread T2(add, 200);
69      std::thread T3(add, 300);
70
71      // Display the initial global variable
72      std::cout << "\nInitial Display:" << std::endl;
73      display();
74
75      // Wait for Thread 1 to finish
76      T1.join();
77
78      // Display the value after Thread 1 finishes
79      std::cout << "\nAfter T1.join():" << std::endl;
80      display();
81
82      // Wait for Thread 2 to finish
83      T2.join();
84
85      // Display the value after Thread 2 finishes
86      std::cout << "\nAfter T2.join():" << std::endl;
87      display();
88
89      // Wait for Thread 3 to finish
90      T3.join();
91
92      // Display the value after Thread 3 finishes
93      std::cout << "\nAfter T3.join():" << std::endl;
94      display();
```

```
96      // Perform experimental analysis
97      std::cout << "\n\nExperimental Analysis:";
98      std::cout << "\n-----------------------------------------------";
99
100     // Test multithreading three times
101     testMultithreading(1);
102     testMultithreading(2);
103     testMultithreading(3);
104
105     std::cout << std::endl;
```

```
106
107     return 0;
108 }
```

Output

```
Multithreading:
-----------------------------------------------
Initial Display:
Global Variable: 600

After T1.join():
Global Variable: 600

After T2.join():
Global Variable: 600

After T3.join():
Global Variable: 600


Experimental Analysis:
-----------------------------------------------
Test Number: 1
Execution Time: 8357 microseconds
Final Value: 600

Test Number: 2
Execution Time: 6701 microseconds
Final Value: 600

Test Number: 3
Execution Time: 8972 microseconds
Final Value: 600
```

Observation	The code successfully demonstrated how ***multithreading*** works in C++ by creating three threads to add different values to a shared ***global variable***. The ***add() function*** added ***100, 200,*** and ***300***, which resulted in a final value of ***600***. The ***std::mutex*** and ***std::lock_guard*** statements were used to protect the global variable while it was being accessed, while the ***join() method*** made the main thread wait until each specified thread finished its task. During the execution, the displayed value remained ***600*** because all three threads were able to finish adding their values before the first display occurred. The ***execution times*** also changed across the three tests, showing that the time needed to complete the task can vary between runs. Through this activity, it became easier to understand how ***multiple threads, shared variables, synchronization,*** and the ***join() method*** work together, although the order of joining threads does not guarantee that the global value will increase after each display.

Asajar's Analysis:	
Theoretical Analysis	The code successfully demonstrated how ***multithreading*** works in C++ by creating three threads that add different values to a shared ***global variable***. The add() function adds ***100, 200,*** and ***300***, while the ***mutex*** and ***lock_guard*** help protect the variable from being accessed by multiple threads at the same time. The ***join() method*** makes the main thread wait for each specified thread to finish, but it does not stop the other threads from running while it waits. Because of this, the global variable may already reach ***600*** before the first display happens.
Experimental Analysis	The output showed that the ***global variable*** was already ***600*** during the initial display and remained the same after ***T1.join(), T2.join(),*** and ***T3.join().*** This happened because the three threads were able to finish their additions before the program displayed the value. The three execution-time tests recorded ***8,357, 6,701,*** and ***8,972 microseconds***, while the final value remained ***600*** in every test. The different times showed that the execution time can change depending on thread scheduling and other computer activities.
Analysis and Comparison	The results matched the expected total because adding ***100 + 200 + 300*** gives ***600***. However, the value did not increase after each ***join()*** because the threads could complete their tasks before the displays occurred. The execution times also changed across the three tests, showing that multithreading does not always have a fixed execution time. Through this activity, it became easier to understand how ***threads, shared variables, synchronization,*** and ***join()*** work together, although the actual performance depends on how the tasks are executed.

Godino's Analysis:	
Theoretical Analysis	The program uses ***three independent threads*** to update a shared integer variable. Each thread executes the ***add()*** function with a different argument, and the ***mutex*** provides synchronized access to the shared data. The ***join()*** calls ensure that the main thread waits for the corresponding worker threads to finish before continuing. Since all threads are started before the first display, their operations may finish in any order, and the first displayed value is not guaranteed to be zero or any particular intermediate total.
Experimental Analysis	The observed output reported ***600*** at every display point, confirming that all three additions had completed before the initial value was printed. The repeated tests produced execution times of ***8,357 µs, 6,701 µs,*** and ***8,972 µs***, with a final value of ***600*** each time. The timing differences indicate that thread scheduling and system conditions can affect the measured duration. These measurements describe the behavior of the multithreaded program, but they do not establish a performance advantage because no single-threaded benchmark was included.
Analysis and Comparison	The experimental results were consistent with the expected final sum of ***600,*** and the mutex helped prevent conflicting access to the shared variable. The unchanged values after each ***join()*** illustrate that joining threads individually does not control when the other threads execute. Although the execution time varied across the tests, a proper comparison of performance would require running an equivalent single-threaded version under similar conditions. Therefore, the activity demonstrates the importance of ***thread synchronization and scheduling***, as well as the need for a fair benchmark when evaluating performance.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Screenshot of the Code	**My Code: (main_PartC.cpp)** SUPPLEMENTARY > main_PartC.cpp > ... 1 #include <iostream> 2 #include <vector> 3 #include <thread> 4 #include <functional> 5 #include <chrono> 6 #include <random> 7 #include <algorithm> 8 9 // Merge two sorted parts of the array 10 void merge(std::vector<int>& arr, int left, int mid, int right){

```
11
12      std::vector<int> temp;
13      int i = left;
14      int j = mid + 1;
15
16      // Compare and merge elements in sorted order
17      while (i <= mid && j <= right){
18          if (arr[i] <= arr[j])
19              temp.push_back(arr[i++]);
20          else
21              temp.push_back(arr[j++]);
22      }
23
24      // Copy the remaining elements
25      while (i <= mid)
26          temp.push_back(arr[i++]);
27
28      while (j <= right)
29          temp.push_back(arr[j++]);
30
31      // Copy the merged elements back to the array
32      for (int k = 0; k < static_cast<int>(temp.size()); k++)
33          arr[left + k] = temp[k];
34  }
35
36  // Sort the array using Multithreaded Merge Sort
37  void mergeSort(std::vector<int>& arr, int left, int right){
38
39      if (left >= right)
40          return;
41
42      int mid = left + (right - left) / 2;
43
44      // Use multithreading for larger sections
45      if (right - left >= 1000){
```

```
46
47          // Sort the left half using a separate thread
48          std::thread T1(
49              mergeSort, std::ref(arr), left, mid
50          );
51
52          // Sort the right half using the current thread
53          mergeSort(arr, mid + 1, right);
54
55          // Wait for the left half to finish
56          T1.join();
57      }
58      else{
59
```

```
60          // Sort smaller sections without creating threads
61          mergeSort(arr, left, mid);
62          mergeSort(arr, mid + 1, right);
63      }
64
65      // Merge the sorted halves
66      merge(arr, left, mid, right);
67  }
68
69  // Display the first 10 elements of the array
70  void displayArr(const std::vector<int>& arr){
71
72      int limit = std::min(10, static_cast<int>(arr.size()));
73
74      for (int i = 0; i < limit; i++){
75          std::cout << arr[i] << " ";
76
77          // Display five values per line
78          if ((i + 1) % 5 == 0)
79              std::cout << std::endl;
80      }
81
82      if (limit % 5 != 0)
83          std::cout << std::endl;
84  }
85
86  // Test the execution time of Multithreaded Merge Sort
87  void testMergeSort(int testNumber, const std::vector<int>& originalArr){
88
89      // Create a copy of the original array
90      std::vector<int> testArr = originalArr;
91
92      // Start the timer
93      auto start = std::chrono::steady_clock::now();
94
```

```
95      // Sort the array
96      mergeSort(testArr, 0, static_cast<int>(testArr.size()) - 1);
97
98      // Stop the timer
99      auto end = std::chrono::steady_clock::now();
100
101     // Calculate the execution time
102     auto duration =
103         std::chrono::duration_cast<std::chrono::microseconds>(
104             end - start
105         );
106
107     // Display the test number
108     std::cout << "\nTest Number: " << testNumber << std::endl;
```

```
109
110     // Display the execution time
111     std::cout << "Execution Time: " << duration.count()
112               << " microseconds" << std::endl;
113
114     // Check if the array is sorted correctly
115     bool sorted = std::is_sorted(testArr.begin(), testArr.end());
116
117     std::cout << "Array Sorted Correctly: " << (sorted ? "Yes" : "No")
118               << std::endl;
119 }
120
121 int main(){
122
123     const int SIZE = 100000;
124     std::vector<int> arr(SIZE);
125
126     // Generate random numbers for the array
127     std::mt19937 gen(42);
128     std::uniform_int_distribution<int> dist(1, 1000000);
129
130     for (int i = 0; i < SIZE; i++)
131         arr[i] = dist(gen);
132
133     // Preserve the original array for experimental testing
134     std::vector<int> originalArr = arr;
135
136     // Display the algorithm name
137     std::cout << "\nMultithreaded Merge Sort:" << std::endl;
138     std::cout << "-------------------------------------------"
139               << std::endl;
140
141     // Display the original array
142     std::cout << "\nOriginal Array (first 10 values):" << std::endl;
143     std::cout << "-------------------------------------------"
```

```
144               << std::endl;
145     displayArr(arr);
146
147     // Sort the array using Multithreaded Merge Sort
148     mergeSort(arr, 0, static_cast<int>(arr.size()) - 1);
149
150     // Display the sorted array
151     std::cout << "\nSorted Array (first 10 values):" << std::endl;
152     std::cout << "-------------------------------------------"
153               << std::endl;
154     displayArr(arr);
155
```

```cpp
    // Check if the array is sorted correctly
    bool sorted = std::is_sorted(arr.begin(), arr.end());

    std::cout << "\nArray Sorted Correctly: " << (sorted ? "Yes" : "No")
              << std::endl;

    // Perform experimental analysis
    std::cout << "\n\nExperimental Analysis:" << std::endl;
    std::cout << "----------------------------------------"
              << std::endl;

    // Display the input size
    std::cout << "Input Size: " << SIZE << std::endl;

    // Test the algorithm three times using the same input
    testMergeSort(1, originalArr);
    testMergeSort(2, originalArr);
    testMergeSort(3, originalArr);

    std::cout << std::endl;

    return 0;
}
```

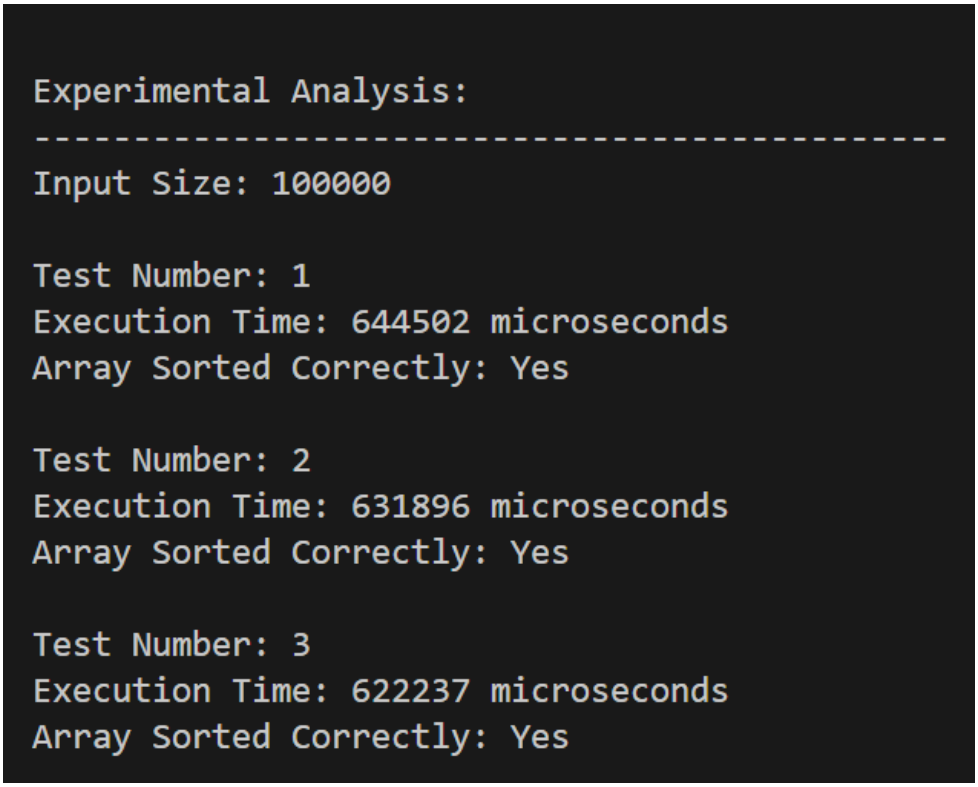
Output

```
Multithreaded Merge Sort:
------------------------------------------------

Original Array (first 10 values):
------------------------------------------------
374541 796543 950715 183435 731994
779691 598659 596851 156019 445833

Sorted Array (first 10 values):
------------------------------------------------
6 10 12 17 21
31 49 53 68 94

Array Sorted Correctly: Yes
```

	
Observation	The code successfully demonstrated how ***multithreaded Merge Sort*** works in C++ by sorting an array containing ***100,000 random integers***. The original array contained unsorted values, while the sorted array displayed the smallest values in ascending order. The ***std::is_sorted()*** function confirmed that the array was sorted correctly during the main execution and all three experimental tests. The recorded execution times were ***644,502, 631,896,*** and ***622,237 microseconds***, with an average of approximately ***632,878 microseconds***. The execution time became slightly lower in each test, showing that the recorded performance can vary depending on ***thread scheduling*** and computer resources. Through this activity, a better understanding of how ***Merge Sort, multithreading,*** and the ***join() method*** work together to sort a large amount of data was gained.

Asajar's Analysis:	
Theoretical Analysis	The ***Merge Sort*** algorithm follows the ***divide-and-conquer*** strategy by splitting the array into smaller parts, sorting each part, and combining them in the correct order. In this code, ***multithreading*** allows one half to be sorted by a separate thread while the other half is handled by the main thread. The ***join()*** method makes sure that the separate thread finishes before both parts are merged together. This helps show how two parts of the sorting process can be handled at the same time.
Experimental Analysis	The program successfully sorted ***100,000 integers***, and the ***std::is_sorted()*** function confirmed that the results were correct in all three tests. The recorded execution times were ***644,502, 631,896,*** and ***622,237 microseconds***, with an average of about ***632,878 microseconds***. The third test had the shortest execution time, while the first test took the longest. These results show that the running time changed slightly during the experiments.
Analysis and Comparison	The results showed that the ***multithreaded Merge Sort*** worked correctly because all three tests produced a sorted array. The execution time also became slightly lower in each test, but this does not automatically mean that the algorithm became faster because of multithreading. The computer's resources and thread scheduling may affect the results. A comparison with a ***single-threaded Merge Sort*** using the same array would help determine if using multiple threads actually improves the sorting speed.

Godino's Analysis:	
Theoretical Analysis	***Merge Sort*** uses a divide-and-conquer approach, where the input is repeatedly divided until smaller sections can be sorted and merged. The program introduces ***concurrency*** by running the left half in a separate thread while the main thread processes the right half. The ***join()*** operation synchronizes the two tasks before the merge step begins, preventing the program from combining the sections before both recursive calls have completed.
Experimental Analysis	The experiment used an input size of ***100,000 elements*** and completed three trials. The measured execution times were ***644,502 µs*** for ***Test 1***, ***631,896 µs*** for ***Test 2***, and ***622,237 µs*** for ***Test 3***. The average was approximately ***632,878 µs***, or ***632.88 ms***. All trials returned ***"Array Sorted Correctly: Yes,"*** confirming that the algorithm maintained the correct ascending order throughout the repeated tests.
Analysis and Comparison	The experimental results support the correctness of the implementation, while the small differences in execution time indicate normal variation between runs. However, the measurements only describe the ***multithreaded implementation*** and cannot establish a speed advantage over a sequential approach. Thread ***creation*** and ***synchronization*** also introduce overhead, which may reduce the benefits of parallel execution. Therefore, measuring a

	single-threaded version under the same conditions would provide a fairer basis for evaluating the performance of multithreaded Merge Sort.

Asajar's Conclusion:

The activity gave a better understanding of how multithreading works in C++ and how it can be used to perform different tasks through multiple threads. For the lessons learned, concurrency allows tasks to make progress during overlapping periods, while parallelism means that tasks are executed at the same time. The std::thread creates a separate thread, while join() makes the main thread wait for the specified thread to finish. The use of mutex and lock_guard also helps protect shared variables from problems when accessed by multiple threads. For the procedure, the codes were created and tested using simple printing tasks, shared variables, and the Merge Sort algorithm. The important parts, such as print(), add(), mergeSort(), join(), and std::is_sorted(), helped show how threads work and how the program checks if the results are correct. For the supplementary activity, the advantages and disadvantages of multithreading were discussed, together with its uses and the differences between concurrency and parallelism.

One difficulty faced was understanding why the global variable already reached 600 during the first display, even before all the join() statements were called. Another difficulty was understanding how multiple threads work together in Merge Sort and why the execution times changed during the three tests. Even with these difficulties, the programs produced correct results, and the 100,000 integers were successfully sorted in all three tests. Overall, the activity was done well because the codes worked correctly and the results were analyzed, but there are still areas for improvement in understanding thread scheduling, synchronization, and execution time comparisons. More practice with multithreaded algorithms and comparing them with single-threaded versions can help improve the understanding of how threads affect program performance and when using multithreading is actually useful.

Godino's Conclusion:

In this activity, I learned about multithreading and making a program perform different tasks at the same time. Through the Merge Sort algorithm, I learned how to divide a large array into smaller parts, sort them separately, and combine them to get the final sorted result. The use of threads and the join() function also helped explain how the program manages its tasks and makes sure that the sorting parts are finished before merging them. In the supplementary activity, I understood that multithreading can help improve the performance of a program, especially when working with a large amount of data, but it can also use more computer resources because of creating threads. The result also showed that the array can still be sorted correctly when the tasks are divided properly. Overall, the activity was helpful in understanding how multithreading works and how it can be applied to sorting algorithms. However, there is still room for improvement in comparing the execution time of single-threaded and multithreaded Merge Sort to know if using multiple threads really makes the program faster.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.